

Unit -3 Software Estimation Techniques [7 Hrs.]

Software Effort Estimation:

Problems with over and under estimations,

Basis of software Estimating,

Software effort estimation techniques,

- expert Judgment,
- Estimating by analogy.
- Bottoms-up estimating,
- Top-down approach and
- parametric models.

Software Effort Estimation

Software effort estimation is a critical process in project management, particularly in the field of software development. It involves predicting the amount of time, resources, and work required to complete a software project successfully. Accurate effort estimation is essential for effective project planning, budgeting, and resource allocation. Here are key aspects of software effort estimation:

Importance of Software Effort Estimation:

- Effort estimation is crucial as it sets the foundation for project planning and control. It helps answer questions such as "How long will the project take?" and "What resources are needed?"

Problems with Overestimation and Underestimation:

- Overestimating the effort can lead to unnecessary resource allocation and extended project timelines, impacting budgets and competitiveness.
- Underestimating the effort can result in missed deadlines, scope creep, and lower quality deliverables, affecting customer satisfaction.

Basis of Software Estimating:

- Estimations are based on a clear understanding of project scope, objectives, and requirements.
- Estimators need to consider factors like project complexity, team expertise, and historical data.

Software Effort Estimation Techniques:

- There are several techniques and methodologies used for software effort estimation, each with its strengths and limitations.
- The choice of technique depends on the project's nature, scope, and available data.

Problems with Over and Under Estimations

Estimating the effort and resources required for a software project is a challenging task. It's crucial to strike the right balance in estimation to ensure project success. However, both overestimation and underestimation can lead to significant problems:

Problems with Overestimation:

- **a. Resource Allocation Issues:**

- Overestimating the effort required can lead to the allocation of excessive resources, such as personnel, time, and budget.
- This misallocation of resources can strain the project's budget, result in underutilized team members, and negatively impact resource availability for other projects.

- **b. Increased Costs:**

- Overestimations can inflate project costs, leading to financial inefficiency.
- Allocating more resources than necessary incurs unnecessary expenses, affecting the project's financial performance.

- **c. Delays in Project Delivery:**

- Overestimation can result in setting overly conservative project schedules.
- Extended project timelines may lead to missed market opportunities or delayed product releases, impacting the project's competitiveness.

- **d. Stakeholder Dissatisfaction:**

- Overestimating project efforts can disappoint stakeholders who expected faster results.
- Dissatisfied stakeholders may lose trust in the project team's capabilities, impacting future collaborations.

Problems with Over and Under Estimations

Estimating the effort and resources required for a software project is a challenging task. It's crucial to strike the right balance in estimation to ensure project success. However, both overestimation and underestimation can lead to significant problems:

Problems with Underestimation:

- **a. Missed Deadlines:**
 - Underestimation is a common cause of missed project deadlines.
 - Unrealistic timeframes may lead to rushed development, lower quality deliverables, or incomplete features.
- **b. Scope Creep:**
 - When project effort is underestimated, scope creep often occurs as new requirements are added during the project.
 - Scope changes can lead to additional work and further project delays.
- **c. Quality Compromises:**
 - Underestimation may force project teams to cut corners or skip crucial quality assurance processes to meet unrealistic deadlines.
 - Lower quality deliverables can lead to increased post-release issues and reduced customer satisfaction.
- **d. Burnout and Stress:**
 - Team members may experience burnout and stress due to the pressure of meeting unattainable goals.
 - Burnout can lead to decreased productivity and morale among project teams.

Basis of Software Estimating

Software estimating is the process of predicting the effort, resources, and time required to complete a software project. It serves as a critical foundation for project planning, budgeting, and management. The basis of software estimating involves several key factors:

1. Project Scope and Objectives:

Estimations start with a clear understanding of the project's scope and objectives. What is the project expected to deliver, and what are its goals? A well-defined scope is essential for accurate estimations.

2. Requirement Analysis:

A comprehensive analysis of project requirements is crucial. Estimators need to examine the specific functionalities, features, and deliverables that the software must provide. Incomplete or vague requirements can lead to estimation inaccuracies.

3. Historical Data:

Historical data from past software projects can be a valuable reference point for estimation. This data provides insights into the effort and resources required for similar projects, helping in making informed estimations.

4. Expertise and Experience:

Expert judgment plays a significant role in software estimating. Experienced professionals and domain experts provide insights based on their knowledge and familiarity with similar projects. Their input is essential for refining estimates.

5. Complexity Assessment:

Assessing the complexity of the project is crucial. Complex projects typically require more effort and resources. Estimators need to evaluate factors like technology, project size, and novel requirements that may increase complexity.

Basis of Software Estimating (Cont.)

6. Risk Evaluation:

Estimators should consider potential risks that can impact the project's duration and resource requirements. These risks may include changes in project scope, technical challenges, and external factors like market conditions.

7. Resource Availability:

Availability of human resources, hardware, and software tools must be considered. Resource constraints can affect project timelines and effort estimations.

8. Methodologies and Tools:

The choice of estimation methodologies and tools also contributes to the basis of software estimating. Different techniques, such as expert judgment, parametric models, or Agile estimation practices, can be applied based on the project's characteristics.

9. Continuous Refinement:

Software estimating is an iterative process. Estimations should be refined as the project progresses and more information becomes available. Continuous monitoring and adjustment are essential for accurate estimations.

10. Stakeholder Involvement:

Engaging stakeholders, including clients and end-users, in the estimation process can provide valuable insights and ensure alignment with project objectives.

Software Effort Estimation Techniques

Estimating the effort required for a software project is a critical aspect of project management. Various techniques and methodologies are used to determine the necessary resources, time, and budget. Here are some common software effort estimation techniques:

1. Expert Judgment:

Expert judgment involves seeking input from experienced professionals, domain experts, and project stakeholders.

Experts provide insights into the effort required based on their knowledge and experience with similar projects.

2. Estimating by Analogy:

Estimating by analogy relies on historical project data to estimate the effort required for the current project.

This technique assumes that projects with similar characteristics will require similar levels of effort.

3. Bottoms-Up Estimating:

Bottoms-up estimating involves estimating individual components or tasks and then aggregating these estimates to determine the overall project effort.

It offers a detailed and granular view of the project's requirements.

4. Top-Down Approach:

The top-down approach starts with a high-level estimate for the entire project and then breaks it down into smaller components.

It is a useful technique when you have limited details at the start of a project.

5. Parametric Models:

Parametric models use mathematical formulas and statistical data to estimate project parameters.

These models are based on historical data and can provide relatively accurate estimations when data is available.

Software Effort Estimation Techniques

6. Function Point Analysis (FPA):

FPA is a method that measures the functionality provided by software.

It quantifies the software's functionality and complexity, helping in estimating project size and effort.

7. COCOMO (Constructive Cost Model):

COCOMO is a systematic model that provides a structured approach to estimating software development effort.

It offers different versions, such as Basic COCOMO, Intermediate COCOMO, and Detailed COCOMO, each suited for projects with varying levels of complexity and data.

8. Three-Point Estimation:

Three-point estimation combines pessimistic, optimistic, and most likely estimates to calculate the expected value.

It helps mitigate estimation bias and provides a more accurate estimate.

9. Agile Estimation Techniques:

Agile methodologies, like Scrum, employ techniques such as Planning Poker and Story Points to estimate work during iterative development.

These techniques involve team collaboration and relative sizing to estimate effort.

10. Estimation Tools:

Estimation tools and software automate the estimation process.

They use historical data, parametric models, and other techniques to provide estimations quickly and accurately.

Choosing the right estimation technique depends on the project's characteristics, available data, and the preferences of the project team. A combination of these techniques may be used to ensure accurate and reliable software effort estimates.

Software Effort Estimation Techniques

Expert Judgment in Software Effort Estimation

Expert judgment is a significant and valuable component of software effort estimation. It involves seeking input and insights from experienced professionals, domain experts, and stakeholders who possess knowledge and expertise relevant to the software project. Here's a closer look at how expert judgment contributes to the estimation process:

1. Expertise-Based Insights:

Experts offer insights and judgments based on their knowledge and experience in the field. They can assess the project's complexity, requirements, and potential challenges.

2. Project-Specific Knowledge:

Experts often have a deep understanding of the specific project and its unique characteristics. Their familiarity with project details is essential for accurate estimation.

3. Identification of Risks:

Experts can identify potential risks and uncertainties that may impact the project's effort and timeline. Their input helps in proactive risk management.

4. Contextual Considerations:

Experts take into account the broader context, such as industry trends, technology advancements, and market conditions, which can affect the project's effort estimation.

5. Bias Mitigation:

Expert judgment helps mitigate biases that can influence estimates. It provides a well-informed and balanced perspective on the project's requirements and challenges.

Software Effort Estimation Techniques

Expert Judgment in Software Effort Estimation (Cont.)

6. Collaboration and Consensus:

Expert judgment often involves collaborative efforts, with multiple experts providing their input.

This collaborative approach fosters consensus and ensures that various aspects of the project are thoroughly considered.

7. Realistic Expectations:

Experts can provide a reality check on the project's feasibility, helping the team set realistic expectations for effort, timelines, and resource requirements.

8. Continuous Refinement:

Expert judgment is not a one-time process. It evolves as the project progresses and more information becomes available.

Experts continuously refine their estimates based on changing project dynamics.

9. Transparency and Communication:

Expert judgment encourages transparency and open communication within the project team.

It allows for the sharing of knowledge and the alignment of team members' perspectives.

Software Effort Estimation Techniques

Bottoms-Up Estimating in Software Effort Estimation

Bottoms-up estimating is a detailed and granular approach used in software effort estimation. It involves breaking down a project into its individual components, estimating the effort required for each component, and then aggregating these estimates to determine the overall project effort. Here's a closer look at how bottoms-up estimating works:

1. Task-Level Estimation:

In bottoms-up estimating, the project is divided into specific tasks, sub-tasks, or work packages. Each of these components is estimated individually.

2. Estimating Individual Components:

Estimators assess each task's requirements, complexity, and other relevant factors to determine the effort required. These efforts may include coding, testing, documentation, and other activities.

3. Resource Allocation:

Estimators allocate resources, including team members and time, to each task based on their estimates. This allocation takes into account the project schedule and resource availability.

4. Data-Driven Approach:

Bottoms-up estimating relies on real data and concrete information. It provides a highly data-driven and precise approach to effort estimation.

5. Aggregating Estimates:

After estimating each individual task, the estimates are aggregated to determine the overall project effort. This summation provides a comprehensive estimate for the entire project.

Software Effort Estimation Techniques

Bottoms-Up Estimating in Software Effort Estimation (Cont.)

6. Detailed Project View:

Bottoms-up estimating offers a detailed view of the project's requirements, providing a clear understanding of the work to be done.

7. Risk Assessment:

Because of its granularity, this method allows for a more accurate assessment of project risks, as it uncovers potential challenges within specific project components.

8. Resource Management:

Resource allocation and task dependencies are effectively managed in bottoms-up estimating. This ensures that resources are optimally utilized throughout the project.

9. Continuous Monitoring:

As the project progresses, bottoms-up estimating allows for continuous monitoring and adjustment of estimates. Any changes or deviations from the original plan can be addressed at the task level.

10. Limitations:

While bottoms-up estimating provides a highly detailed estimation, it can be time-consuming, especially for large projects. It also requires a well-defined project scope and detailed task breakdown.

Software Effort Estimation Techniques

Top-Down Approach in Software Effort Estimation

The top-down approach is a method used in software effort estimation that begins with a high-level estimation for the entire project and then gradually breaks it down into smaller components. This approach allows for a broad view of the project's scope before delving into the details. Here's a closer look at how the top-down approach works:

1. High-Level Project Estimation:

The top-down approach starts with an initial high-level estimation for the entire software project. This estimation provides an overarching view of the project's effort requirements.

2. Broad Scope Understanding:

At this stage, estimators focus on understanding the project's broad scope, objectives, and high-level requirements. It is a high-level, abstract view of the project.

3. Component Identification:

Once the high-level estimation is established, estimators identify the major components, milestones, or phases that make up the project. These components represent the top levels of the project's structure.

4. Gradual Decomposition:

Estimators then gradually decompose each major component into smaller, more detailed components. This decomposition process continues until a sufficient level of detail is reached.

5. Component-Level Estimations:

Estimations for the smaller components are made, typically in a more detailed manner, once they are identified.

Software Effort Estimation Techniques

Top-Down Approach in Software Effort Estimation (Cont.)

6. Flexibility and Adaptability:

The top-down approach provides flexibility in estimation as it allows for adjustments and refinements as more information becomes available and the project progresses.

7. Initial Planning and High-Level Decision-Making:

This approach is particularly useful in the early stages of project planning and decision-making, allowing project teams to get an initial sense of the project's scale and resource needs.

8. Requirement Elicitation:

The top-down approach aids in requirement elicitation as it encourages discussions about the project's high-level objectives and expected outcomes.

9. Limitations:

While it offers an efficient high-level view, the top-down approach may require further detail and precision for complex projects, which can be achieved through subsequent detailed estimating techniques.

Software Effort Estimation Techniques

Parametric Models in Software Effort Estimation

Parametric models are systematic and mathematical approaches to software effort estimation. These models rely on historical data, formulas, and statistical techniques to estimate project parameters. Here's an in-depth look at how parametric models work:

1. Historical Data Analysis:

Parametric models use historical project data as a foundation for estimation. This data includes details about previous software projects, such as project size, effort, duration, and outcomes.

2. Quantitative Estimation:

Parametric models are quantitative in nature. They use mathematical formulas and statistical analysis to estimate project parameters.

3. Input Parameters:

Estimators provide specific input parameters to the model. These parameters can include project size (e.g., lines of code or function points), complexity, and team productivity metrics.

4. Data Calibration:

The model calibrates the input parameters based on historical data and applies mathematical formulas to produce estimations.

5. Model Types:

Various parametric models exist, such as the COCOMO (Constructive Cost Model) and SLIM (Software Lifecycle Management) model. These models are tailored for different types of projects and complexities.

Software Effort Estimation Techniques

Parametric Models in Software Effort Estimation (Cont.)

6. Project Complexity Considerations:

Parametric models account for project complexity, which includes factors like technology, requirements, and potential risks that can influence the estimation.

7. Resource Allocation:

The models estimate resource allocation, including team members and their availability, as well as the project timeline and budget.

8. Multiple Variants:

Parametric models often come in multiple variants, each suitable for different types of software projects. For example, COCOMO offers Basic, Intermediate, and Detailed variants.

9. Data-Driven Accuracy:

Parametric models are known for their data-driven accuracy. They provide estimations based on the historical performance of similar projects, making them reliable.

10. Continuous Refinement:

Estimations from parametric models can be refined as more project-specific data becomes available or as the project progresses.